

SIM_NN Project Explanation

*Neural-network control of the phase shifts of a Stacked Intelligent Metasurface (SIM) — a summary document based on the file **SIM_NN_Phase_bis.py** and the related files in the **SIM_NN** folder.*

1. General background

A SIM (Stacked Intelligent Metasurface) is a stack of L reconfigurable metasurfaces placed between a base station (BS) and K user equipments (UEs), operating at millimeter-wave frequencies (28 GHz). Each metasurface is made of N meta-atoms whose phase shift can be digitally controlled (quantized over N_{bitQ} bits). The goal of the script is to train a neural network that, given the channel matrix H , predicts the phase shifts to apply to every meta-atom of every layer in order to maximize the spectral efficiency (SE) of the worst-off user (max-min fairness criterion).

2. Structure of SIM_NN_Phase_bis.py

2.1 Key functions

- **NormP_Q(sig)**: quantizes a phase signal over N_{bitQ} bits (2 bits here, i.e. 4 discrete levels) via `tf.quantization.fake_quant_with_min_max_args`.
- **CustomLoss(y_true, y_pred)**: the physics-based loss function, the core of the model. It reconstructs the complex channel matrix from y_{true} (concatenated real/imaginary parts), quantizes the predicted phases, computes the complex phase shift $\exp(j\theta)$, propagates the signal through the L layers of the SIM (matrix G = cumulative product of the diagonal phase matrices and the transmission matrices W_l), then computes the rate ($\log_2(1+\text{SINR})$) of each user. The returned loss is the negative of the minimum rate averaged over the batch (max-min fairness).

2.2 Physical parameters

Hard-coded at the top of the script: $K=4$ users, $M=K$ BS antennas, $L=10$ metasurfaces, $N = N_x \times N_y = 7 \times 7 = 49$ meta-atoms per layer, $N_{\text{bitQ}}=2$ phase-quantization bits, $f=28$ GHz, power budget, thermal noise (σ_{noise}), and a path-loss model (exponent $n_{\text{bar}}=3.5$).

2.3 Geometry and transmission matrices

From the physical parameters, the script computes the distances between meta-atoms (D_{nn}), between consecutive layers (D_l), the incidence angles, then builds the inter-layer transmission matrices **WI** and the BS $\rightarrow$ first-layer matrix **W1** (functions defined in `functions.py`). These matrices physically model the wave propagation through the stack of metasurfaces.

2.4 Data generation

One (or several) channel realization(s) H are generated randomly (Rayleigh fading, spatially correlated via R_{nn}). The channel is reshaped into a concatenated real/imaginary form to serve both as the input AND as the "target" (y_{true}) of the Keras model: this is a trick to pass the channel matrix through to the custom loss function, which reconstructs it via reshape. Training is therefore **unsupervised** (there are no real labels; the loss is a physics-based objective computed directly).

2.5 Network architecture

A 1D CNN: Conv1D(16 filters, kernel=3, ReLU) → MaxPooling1D → Flatten → Dense(L*N, activation = a custom function mapping to $[0, 2\pi]$). The output represents the L*N phase shifts to apply across all meta-atoms of the SIM.

2.6 Training, saving and evaluation

The model is compiled with the Adam optimizer and CustomLoss, then trained for 1000 epochs (batch_size = btch). The weights of every epoch are stored via a callback, the best epoch (lowest loss) is selected, and the weights plus the channel H are saved to a .mat file. The model is then re-evaluated: the predicted phases are quantized, the per-user rate is recomputed, and a cumulative distribution function (CDF) curve of the spectral efficiency is plotted.

Important note: in this file, Nch = 1 and btch = 1, so a single channel realization is used for training, validation (H2 = H1) and testing (H3 = H1) alike. This is therefore not a true statistical generalization test but rather a convergence check on a single channel instance.

3. How to test the code

- Check dependencies: tensorflow / keras, numpy, scipy, matplotlib.
- Run from the SIM_NN/ folder (the script does from `functions import *`, so `functions.py` must be accessible).
- Launch it with: `python SIM_NN_Phase_bis.py`
- For a quick first test, temporarily lower epochs (e.g. 20-50 instead of 1000) to check there are no errors before running the full training, which can be slow (non-vectorized Python loops inside CustomLoss).
- Expected outputs: model summary printed to the console, a loss curve (train/val), a .mat file containing the channel and weights, and a CDF plot of the spectral efficiency.
- For a more rigorous test: increase Nch (several channel realizations) and generate H2/H3 independently of H1 (currently those lines are commented out in the script).

4. Overview of the files in the SIM_NN folder

File	Role
functions.py	Shared library: SIM geometry (distances, transmission matrices $W/W1$), path-loss, channel generation.
SIM_NN_Phase.py	A version close to the "bis" variant, controlling only the phases ($L=10$), with an optional (commented out) power constraint.
SIM_NN_Phase_bis.py	The reference file for this document: the "bis" variant (a corrected/cleaned iteration), using a 1D CustomLoss.
SIM_NN_Phase_Power.py	Extends the model to two outputs: task1_output (phases) + task2_output (power, via custom_activation).
SIM_NN_Phase_StatCl.py	A variant with $L=5$ (instead of 10), intended to be run with several channel realizations ($Nch > 1$) to test statistical learning.
SIM_NN_Phase_Sys2.py	A variant for a different system configuration (different antenna/user setup).
ChannelEstim_NN.py / ChannelEstim_NN.py	Networks dedicated to channel estimation (likely from pilot signals), distinct from phase optimization.
ANN_SIM_DualTask.py (+ .txt)	A more general "dual-task" model; the .txt is probably a backup / text output associated with it.

In short: `functions.py` is the shared physics module, the `SIM_NN_Phase*.py` files are experimental variants of the same pipeline (phases only, phases+power, other system, other number of layers), while `ChannelEstim_NN*` and `ANN_SIM_DualTask` explore related tasks (channel estimation, multi-task learning).